

outcome as a key and the count as a value, and `self.percentsDict`, which uses each outcome as a key and the percentage of rolls as a value.

In more complex programs, the model could get its data from a database, the internet, or other sources. For example, a `Model` object could maintain stock information, population data, city housing data, temperature readings, and so on.

In this model, the `getRoundsRollsPercents()` method is called by the `View` objects to retrieve all the data at once. However, a model may contain more information than any one view might need. Therefore, different `View` objects can call different methods in the `Model` object to request different information from the same model. To support this, in the sample program I've included a number of additional getter methods (`getNumberOfRounds()`, `getRolls()`, and `getPercents()`) that a programmer could use when building a new `View` object, to get only the data that the new view might want to display.

The View

A `View` object is responsible for displaying data to the user. In our sample program, we have three different `View` objects that display the same underlying information in three different forms; each displays the information within the black rectangle in the window. At startup, and when the user clicks Roll Dice, the controller calls the `update()` method of the current `View` object. All `View` objects then make the same call to the `Model` object to get the current data:

```
nRounds, resultsDict, percentsDict = self.oModel.getRoundsRollsPercents()
```

The `View` object then formats the data in its own way and presents it to the user.

Advantages of the MVC Pattern

The MVC design pattern breaks up responsibility into separate classes that act independently but work collectively. Building the components as distinct classes and minimizing the interactions between the resulting objects allows each individual component to be less complex and less error